

第6章: プロジェクトセットアップ — 書籍管理アプリの土台を作る

いよいよ実際のアプリ開発が始まります！この章では、書籍管理アプリの「土台」（プロジェクトの雛形、フォルダ構成、初期設定）を作ります。

この章で行うこと

ここまでの章で学んだ技術（TypeScript、React、Next.js、Supabase）を組み合わせ、実際のプロジェクトを作成します。料理に例えると、材料を揃えて下さりえをする段階です。

1. Next.js プロジェクトの作成 — `npx create-next-app` コマンド（プロジェクトの雛形を自動生成するコマンド）を実行
2. Tailwind CSS の理解 — Tailwind CSS（テールウィンドCSS：HTMLのクラス名でスタイルを直接指定する CSSフレームワーク）の使い方
3. フォルダ構成の設計 — どのファイルをどこに置くか、プロジェクト全体の設計図を決める
4. Supabase との接続設定 — アプリからデータベースに接続するための設定ファイルを作成
5. 型定義の作成 — 書籍データの「Book型」をTypeScriptで定義

この章を終えると、ブラウザで開発サーバー（Development Server：開発中にアプリを確認するためのローカルサーバー。`http://localhost:3000` でアクセスできる）にアクセスし、ヘッダー付きのトップページが表示される状態になります。

ポイント： この章はコードを書く量が多いですが、一つ一つのファイルの役割を理解しながら進めてください。「なぜこのファイルが必要なのか」を意識すると、後の章で迷わなくなります。

目次

1. Next.js プロジェクトの作成
2. Tailwind CSS の基礎
3. 必要なパッケージのインストール
4. プロジェクト構成の作成
5. Supabase クライアントの設定
6. 型定義ファイルの作成
7. ルートレイアウトの設定
8. 共通ヘッダーコンポーネント

- 9. 開発サーバーの起動と動作確認
- 10. データフローの全体像

1. Next.js プロジェクトの作成

1.1 create-next-app コマンドの実行

ターミナル（コマンドプロンプト、PowerShell、または VS Code のターミナル）を開き、プロジェクトを作成したいディレクトリに移動してから、以下のコマンドを実行します。

```
npx create-next-app@latest book-management
```

npx とは？ npx は npm 5.2 以降に同梱されているコマンドで、パッケージをグローバルインストールせずに一時的に実行できます。**create-next-app@latest** は「最新版の create-next-app を使う」という意味です。

コマンドを実行すると、いくつかの質問が対話形式で表示されます。以下のように回答してください。

1.2 各選択肢の詳細解説

```
Would you like to use TypeScript? ... Yes
Would you like to use ESLint? ... Yes
Would you like to use Tailwind CSS? ... Yes
Would you like your code inside a `src/` directory? ... Yes
Would you like to use App Router? (recommended) ... Yes
Would you like to use Turbopack for next dev? ... Yes
Would you like to customize the import alias (@/* by default)? ... No
```

それぞれの選択肢が何を意味するのか、なぜその回答をするのかを詳しく見ていきましょう。

質問	回答	理由
TypeScript	Yes	型安全性により、変数や関数の引数に間違った値を渡すミスをコンパイル時に検出できます。特にチーム開発やアプリが大規模になったときに威力を発揮します。本チュートリアルでは、Book 型などを定義してデータの構造を明確にします。
ESLint	Yes	JavaScript/TypeScript のコードを静的解析し、バグの原因になりやすい書き方やスタイルの不統一を自動検出します。例えば、未使用の変数がある場合に警告を出してくれます。
Tailwind CSS	Yes	ユーティリティファーストの CSS フレームワークです。HTML (JSX) の中にクラス名を書くだけでスタイリングできるため、CSS ファイルを別途管理する必要がありません。詳しくは次のセクションで解説します。
<code>src/</code> directory	Yes	ソースコードを <code>src/</code> ディレクトリにまとめることで、設定ファイル (<code>package.json</code> , <code>next.config.ts</code> など) とソースコードが明確に分離されます。プロジェクトのルートがすっきりし、見通しがよくなります。
App Router	Yes	Next.js 13 以降で導入された新しいルーティング方式です。ファイルシステムベースのルーティング、Server Components、レイアウトのネストなど、モダンな機能が使えます。Pages Router (旧方式) より推奨されています。
Turbopack	Yes	Rust で書かれた高速なバンドラーです。開発サーバーの起動やホットリロード (コード変更時の自動反映) が従来の Webpack より大幅に高速になります。
Import alias	No (デフォルト)	デフォルトの <code>@/*</code> エイリアスをそのまま使います。 <code>@/components/Header</code> のように <code>src/</code> ディレクトリ内のファイルを短いパスでインポートできます。

1.3 なぜ Tailwind CSS を選ぶのか

CSS のスタイリング手法にはさまざまな選択肢があります。以下の比較表で、それぞれの特徴を整理します。

項目	Tailwind CSS	CSS Modules	styled-components
スタイルの書き場所	JSX のクラス名に直接記述	<code>.module.css</code> ファイルに記述	JavaScript ファイル内に記述
学習コスト	クラス名を覚える必要がある (慣れると高速)	通常の CSS と同じ知識で書ける	CSS + JavaScript の知識が必要
ファイル数	CSS ファイル不要 (少ない)	コンポーネントごとに CSS ファイルが必要 (多い)	CSS ファイル不要 (少ない)
名前の衝突	なし (ユーティリティクラスのため)	なし (自動でスコープされる)	なし (自動でスコープされる)
バンドルサイズ	未使用クラスが自動削除される (小さい)	使った分だけ (中程度)	ランタイムで CSS を生成 (やや大きい)
レスポンシブ対応	<code>md:</code> , <code>lg:</code> などのプレフィックスで簡単	メディアクエリを自分で書く	メディアクエリを自分で書く
デザインの一貫性	デフォルトのデザインシステム内蔵	自分で設計する必要がある	自分で設計する必要がある
Server Components 対応	完全対応	完全対応	対応に制限あり (<code>'use client'</code> が必要)
本チュートリアルでの採用	採用	不採用	不採用

Tailwind CSS を選ぶ最大の理由は「開発速度」です。CSS ファイルを作成して行き来する必要がなく、JSX を書きながらそのままスタイリングできます。最初はクラス名を覚えるのに少し時間がかかりますが、慣れると非常に高速に UI を構築できるようになります。

1.4 コマンド実行中のターミナル出力

質問に全て答え終わると、ターミナルに次のような表示が流れます（実際の表示は時期によって少し変わります）。

```
Creating a new Next.js app in /Users/you/projects/book-management.

Using npm.

Initializing project with template: app

Installing dependencies:
- react
- react-dom
- next

Installing devDependencies:
- typescript
- @types/node
- @types/react
- @types/react-dom
- tailwindcss
- postcss
- eslint
- eslint-config-next

added 372 packages, and audited 373 packages in 24s

131 packages are looking for funding
found 0 vulnerabilities
Initialized a git repository.

Success! Created book-management at /Users/you/projects/book-management
```

「added XXX packages」とは?: `npm install` が `node_modules/` フォルダに何個のパッケージを置いたかを示します。React/Next.js本体に加えて、依存パッケージ（依存関係でついてくる関連ライブラリ）と一緒に入るので300〜400個程度になります。これが普通です。

完了したら、表示されたプロジェクトフォルダに `cd` で移動します。

```
cd book-management
```

1.5 生成されるファイルの役割

`create-next-app` を実行すると、以下のファイルとディレクトリが自動生成されます。

book-management/	
node_modules/	... インストールされたパッケージ群
public/	... 静的ファイル (画像など)
file.svg	
globe.svg	
next.svg	
vercel.svg	
window.svg	
src/	
app/	
favicon.ico	... ブラウザのタブに表示されるアイコン
globals.css	... グローバル CSS (Tailwind の設定を含む)
layout.tsx	... ルートレイアウト (全ページ共通の枠)
page.tsx	... トップページ (/ にアクセスしたときの表示)
.eslintrc.json	... ESLint の設定ファイル
.gitignore	... Git で管理しないファイルの指定
next-env.d.ts	... Next.js の TypeScript 型定義 (自動生成)
next.config.ts	... Next.js の設定ファイル
package-lock.json	... パッケージの正確なバージョン記録
package.json	... プロジェクトの設定とパッケージ一覧
postcss.config.mjs	... PostCSS の設定 (Tailwind CSS が使用)
README.md	... プロジェクトの説明
tailwind.config.ts	... Tailwind CSS のカスタマイズ設定
tsconfig.json	... TypeScript の設定ファイル

各ファイルの役割を表にまとめます。

ファイル	役割
<code>package.json</code>	プロジェクト名、使用パッケージ、実行スクリプトなどを定義するファイルです。 <code>npm install</code> はこのファイルを読んでパッケージをインストールします。
<code>tsconfig.json</code>	TypeScript コンパイラの設定ファイルです。 <code>@/*</code> のパスエイリアスもここで定義されています。
<code>next.config.ts</code>	Next.js の動作をカスタマイズする設定ファイルです。画像の外部ドメイン許可などを設定できます。
<code>tailwind.config.ts</code>	Tailwind CSS のカスタマイズ設定ファイルです。カスタムカラーやフォントの追加ができます。
<code>postcss.config.mjs</code>	PostCSS（CSS の変換ツール）の設定です。Tailwind CSS は PostCSS のプラグインとして動作します。
<code>.eslintrc.json</code>	ESLint のルール設定です。 <code>next/core-web-vitals</code> の推奨ルールがデフォルトで有効になっています。
<code>src/app/layout.tsx</code>	すべてのページを囲むルートレイアウトです。 <code><html></code> と <code><body></code> タグを含み、共通のヘッダーやフッターを配置します。
<code>src/app/page.tsx</code>	トップページのコンポーネントです。URL <code>/</code> にアクセスしたときに表示されます。
<code>src/app/globals.css</code>	アプリ全体に適用されるグローバル CSS です。Tailwind CSS のディレクティブ（ <code>@tailwind base;</code> など）が含まれています。

2. Tailwind CSS の基礎

2.1 ユーティリティファーストとは

従来の CSS では、まずクラス名を考え、そのクラスに対してスタイルを定義していました。

```
/* 従来の CSS */
.card {
  background-color: white;
  border-radius: 8px;
  padding: 16px;
  box-shadow: 0 1px 3px rgba(0, 0, 0, 0.1);
}
```

```
<div class="card">...</div>
```

Tailwind CSS では、あらかじめ用意された小さなユーティリティクラスを組み合わせでスタイリングします。

```
<!-- Tailwind CSS -->
<div class="bg-white rounded-lg p-4 shadow-sm">...</div>
```

それぞれのクラスが1つの CSS プロパティに対応しています。

クラス	対応する CSS
<code>bg-white</code>	<code>background-color: white;</code>
<code>rounded-lg</code>	<code>border-radius: 0.5rem;</code>
<code>p-4</code>	<code>padding: 1rem;</code>
<code>shadow-sm</code>	<code>box-shadow: 0 1px 2px rgba(0,0,0,0.05);</code>

ユーティリティファーストのメリット:

- CSS ファイルを作る必要がない (JSX だけで完結する)
- クラス名を考える時間がゼロになる
- デザインシステム (余白・色・サイズ) が統一される
- 使っていないスタイルが自動で削除される (バンドルサイズが小さい)

ユーティリティファーストのデメリット:

- JSX のクラス名が長くなることがある
- 最初はクラス名を覚えるのに時間がかかる

2.2 よく使うクラス一覧

本チュートリアルで頻繁に使うクラスを以下にまとめます。この表をブックマークしておくと、開発中に素早く参照できます。

余白 (Spacing)

クラス	CSS	説明
<code>p-4</code>	<code>padding: 1rem; (16px)</code>	内側の余白 (全方向)
<code>px-4</code>	<code>padding-left: 1rem; padding-right: 1rem;</code>	内側の余白 (左右)
<code>py-2</code>	<code>padding-top: 0.5rem; padding-bottom: 0.5rem;</code>	内側の余白 (上下)
<code>pt-4</code>	<code>padding-top: 1rem;</code>	内側の余白 (上のみ)
<code>pb-4</code>	<code>padding-bottom: 1rem;</code>	内側の余白 (下のみ)
<code>m-4</code>	<code>margin: 1rem;</code>	外側の余白 (全方向)
<code>mx-auto</code>	<code>margin-left: auto; margin-right: auto;</code>	要素を水平方向に中央揃え
<code>mt-8</code>	<code>margin-top: 2rem;</code>	外側の余白 (上のみ)
<code>mb-4</code>	<code>margin-bottom: 1rem;</code>	外側の余白 (下のみ)
<code>space-y-4</code>	子要素間に <code>margin-top: 1rem;</code>	子要素の縦方向の間隔
<code>gap-4</code>	<code>gap: 1rem;</code>	Flex/Grid の子要素間の間隔

数値の法則: Tailwind の数値は `0.25rem` (4px) 単位です。 `p-1` = 4px、`p-2` = 8px、`p-4` = 16px、`p-8` = 32px のように、数値を4倍するとピクセル値になります。

色 (Colors)

クラス	説明
<code>text-gray-900</code>	濃いグレーの文字色（ほぼ黒）
<code>text-gray-600</code>	中間のグレーの文字色
<code>text-gray-400</code>	薄いグレーの文字色
<code>text-blue-600</code>	青い文字色（リンクなど）
<code>text-red-600</code>	赤い文字色（エラーメッセージなど）
<code>text-green-600</code>	緑の文字色（成功メッセージなど）
<code>text-white</code>	白い文字色
<code>bg-white</code>	白い背景色
<code>bg-gray-50</code>	ごく薄いグレーの背景色
<code>bg-gray-100</code>	薄いグレーの背景色
<code>bg-blue-600</code>	青い背景色（ボタンなど）
<code>bg-red-600</code>	赤い背景色（削除ボタンなど）
<code>border-gray-200</code>	薄いグレーのボーダー色
<code>border-gray-300</code>	ボーダー色（フォーム入力欄など）

色の濃さ: 50 が最も薄く、950 が最も濃い色です。100 刻みで用意されています。gray-50（ほぼ白）、gray-500（中間）、gray-900（ほぼ黒）のようなイメージです。

タイポグラフィ (Typography)

クラス	CSS	説明
<code>text-xs</code>	<code>font-size: 0.75rem;</code>	極小テキスト (12px)
<code>text-sm</code>	<code>font-size: 0.875rem;</code>	小さいテキスト (14px)
<code>text-base</code>	<code>font-size: 1rem;</code>	通常テキスト (16px)
<code>text-lg</code>	<code>font-size: 1.125rem;</code>	やや大きいテキスト (18px)
<code>text-xl</code>	<code>font-size: 1.25rem;</code>	大きいテキスト (20px)
<code>text-2xl</code>	<code>font-size: 1.5rem;</code>	見出し (24px)
<code>text-3xl</code>	<code>font-size: 1.875rem;</code>	大見出し (30px)
<code>font-bold</code>	<code>font-weight: 700;</code>	太字
<code>font-semibold</code>	<code>font-weight: 600;</code>	やや太字
<code>font-medium</code>	<code>font-weight: 500;</code>	やや太字 (控えめ)
<code>font-normal</code>	<code>font-weight: 400;</code>	通常の太さ
<code>text-center</code>	<code>text-align: center;</code>	テキスト中央揃え
<code>truncate</code>	<code>overflow: hidden; text-overflow: ellipsis; white-space: nowrap;</code>	テキストが長い場合に「...」で省略

レイアウト（Layout / Flexbox / Grid）

クラス	CSS	説明
<code>flex</code>	<code>display: flex;</code>	Flexbox コンテナにする
<code>flex-col</code>	<code>flex-direction: column;</code>	子要素を縦に並べる
<code>items-center</code>	<code>align-items: center;</code>	交差軸方向に中央揃え
<code>justify-between</code>	<code>justify-content: space-between;</code>	両端に配置
<code>justify-center</code>	<code>justify-content: center;</code>	主軸方向に中央揃え
<code>grid</code>	<code>display: grid;</code>	Grid コンテナにする
<code>grid-cols-1</code>	<code>grid-template-columns: repeat(1, 1fr);</code>	1列のグリッド
<code>grid-cols-2</code>	<code>grid-template-columns: repeat(2, 1fr);</code>	2列のグリッド
<code>grid-cols-3</code>	<code>grid-template-columns: repeat(3, 1fr);</code>	3列のグリッド
<code>w-full</code>	<code>width: 100%;</code>	横幅を親要素いっぱい
<code>max-w-7xl</code>	<code>max-width: 80rem;</code>	最大横幅（1280px）
<code>min-h-screen</code>	<code>min-height: 100vh;</code>	最小高さを画面いっぱい
<code>hidden</code>	<code>display: none;</code>	要素を非表示にする

ボーダー・角丸・影（Borders / Radius / Shadow）

クラス	CSS	説明
<code>border</code>	<code>border-width: 1px;</code>	1px のボーダーを追加
<code>border-2</code>	<code>border-width: 2px;</code>	2px のボーダーを追加
<code>rounded</code>	<code>border-radius: 0.25rem;</code>	少し角丸
<code>rounded-md</code>	<code>border-radius: 0.375rem;</code>	中程度の角丸
<code>rounded-lg</code>	<code>border-radius: 0.5rem;</code>	大きめの角丸
<code>rounded-full</code>	<code>border-radius: 9999px;</code>	完全な丸（バッジやアイコンに）
<code>shadow-sm</code>	小さなシャドウ	控えめな影
<code>shadow</code>	中程度のシャドウ	標準的な影
<code>shadow-lg</code>	大きなシャドウ	目立つ影

状態変化 (Hover / Focus)

クラス	説明
<code>hover:bg-blue-700</code>	マウスを載せたときに背景色を濃い青に変更
<code>hover:text-blue-600</code>	マウスを載せたときに文字色を青に変更
<code>hover:shadow-md</code>	マウスを載せたときに影を追加
<code>focus:outline-none</code>	フォーカス時のデフォルトのアウトラインを消す
<code>focus:ring-2</code>	フォーカス時に2pxのリングを表示
<code>focus:ring-blue-500</code>	フォーカスリングの色を青に設定
<code>transition</code>	CSS トランジションを有効にする (スムーズなアニメーション)
<code>duration-200</code>	トランジション時間を200msに設定
<code>cursor-pointer</code>	マウスカーソルをポインター (指) にする
<code>disabled:opacity-50</code>	無効状態のときに半透明にする
<code>disabled:cursor-not-allowed</code>	無効状態のときにカーソルを「禁止」にする

2.3 レスポンシブデザイン

Tailwind CSS では、ブレイクポイントのプレフィックスを付けることで、画面サイズに応じたスタイルを簡単に適用できます。

プレフィックス	最小幅	対象デバイス
(なし)	0px	すべて (モバイルファースト)
<code>sm:</code>	640px	大きめスマートフォン
<code>md:</code>	768px	タブレット
<code>lg:</code>	1024px	ノートPC
<code>xl:</code>	1280px	デスクトップ
<code>2xl:</code>	1536px	大画面

重要: Tailwind はモバイルファースト設計です。プレフィックスなしのクラスがモバイル (最小画面) のスタイルになり、プレフィックス付きのクラスが「その画面サイズ以上」に適用されます。

実例: レスポンシブなグリッドレイアウト

```
<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
  <div>カード1</div>
  <div>カード2</div>
  <div>カード3</div>
</div>
```

この例では次のように表示が変わります。

画面サイズ	適用されるクラス	表示
0px ~ 767px	<code>grid-cols-1</code>	1列表示（カードが縦に並ぶ）
768px ~ 1023px	<code>md:grid-cols-2</code>	2列表示
1024px ~	<code>lg:grid-cols-3</code>	3列表示

2.4 「このクラスを付けたらこう表示される」具体例

実際にボタンを例にして、クラスを1つずつ追加していくとどう変わるか見てみましょう。

Step 1: テキストだけ（クラスなし）

```
<button>書籍を登録する</button>
```

ブラウザデフォルトのボタンが表示されます。装飾はほぼなく、小さな灰色の枠線と白い背景のボタンです。

Step 2: 背景色と文字色を追加

```
<button className="bg-blue-600 text-white">書籍を登録する</button>
```

青い背景に白い文字のボタンになります。ただし、余白がなくテキストがギリギリです。

Step 3: 余白を追加

```
<button className="bg-blue-600 text-white px-4 py-2">書籍を登録する</button>
```

左右に16px、上下に8pxの余白が入り、見た目が改善されます。

Step 4: 角丸を追加

```
<button className="bg-blue-600 text-white px-4 py-2 rounded-md">書籍を登録する</button>
```

角が丸くなり、モダンな印象になります。

Step 5: ホバーエフェクトとトランジションを追加

```
<button className="bg-blue-600 text-white px-4 py-2 rounded-md hover:bg-blue-700 transition"
  書籍を登録する
</button>
```

マウスを載せると背景色が少し濃くなり、200ms かけてスムーズに変化します。

Step 6: フォーカスリングを追加

```
<button className="bg-blue-600 text-white px-4 py-2 rounded-md hover:bg-blue-700 transition"
  書籍を登録する
</button>
```

Tab キーでフォーカスしたときに、ボタンの周りに青いリングが表示されます。これはアクセシビリティ（キーボード操作対応）のために重要です。

完成形: 本チュートリアルで使うボタンスタイル

```
<button className="bg-blue-600 text-white px-4 py-2 rounded-md font-medium hover:bg-blue-700"
  書籍を登録する
</button>
```

3. 必要なパッケージのインストール

プロジェクトのディレクトリに移動してから、必要なパッケージをインストールします。

```
cd book-management
npm install @supabase/supabase-js
```

3.1 各パッケージの役割

パッケージ名	役割	説明
<code>@supabase/supabase-js</code>	Supabase クライアント	Supabase（データベース、認証、ストレージ）と通信するための公式 JavaScript/TypeScript ライブラリです。このパッケージを使って、ブラウザから直接 Supabase の PostgreSQL データベースにデータの読み書きができます。
<code>next</code> （プリインストール済み）	Next.js フレームワーク	React ベースのフルスタックフレームワークです。ルーティング、サーバーサイドレンダリング、API ルートなどの機能を提供します。
<code>react</code> （プリインストール済み）	React ライブラリ	UI コンポーネントを構築するためのライブラリです。 <code>useState</code> や <code>useEffect</code> などのフック（Hook）を使って、動的な UI を作成します。
<code>react-dom</code> （プリインストール済み）	React DOM	React コンポーネントをブラウザの DOM（画面）にレンダリングするためのライブラリです。
<code>typescript</code> （プリインストール済み）	TypeScript	JavaScript に型システムを追加した言語です。開発時の型チェックに使われます。
<code>tailwindcss</code> （プリインストール済み）	Tailwind CSS	ユーティリティファーストの CSS フレームワークです。 <code>create-next-app</code> で Tailwind CSS を選択した場合、自動的にインストールされます。
<code>eslint</code> （プリインストール済み）	ESLint	コードの品質チェックツールです。バグの原因になりやすいパターンを検出します。

`@supabase/supabase-js` だけインストールすれば十分な理由: 本チュートリアル of 書籍管理アプリでは、Supabase との通信がメインの外部依存です。UI は Tailwind CSS（プリインストール済み）で構築し、フォーム処理は React の標準機能で実現します。追加のUIライブラリやフォームライブラリは使用しないため、シンプルな構成を保てます。

3.2 パッケージがインストールされたことの確認

`package.json` を開き、`dependencies` に `@supabase/supabase-js` が追加されていることを確認します。

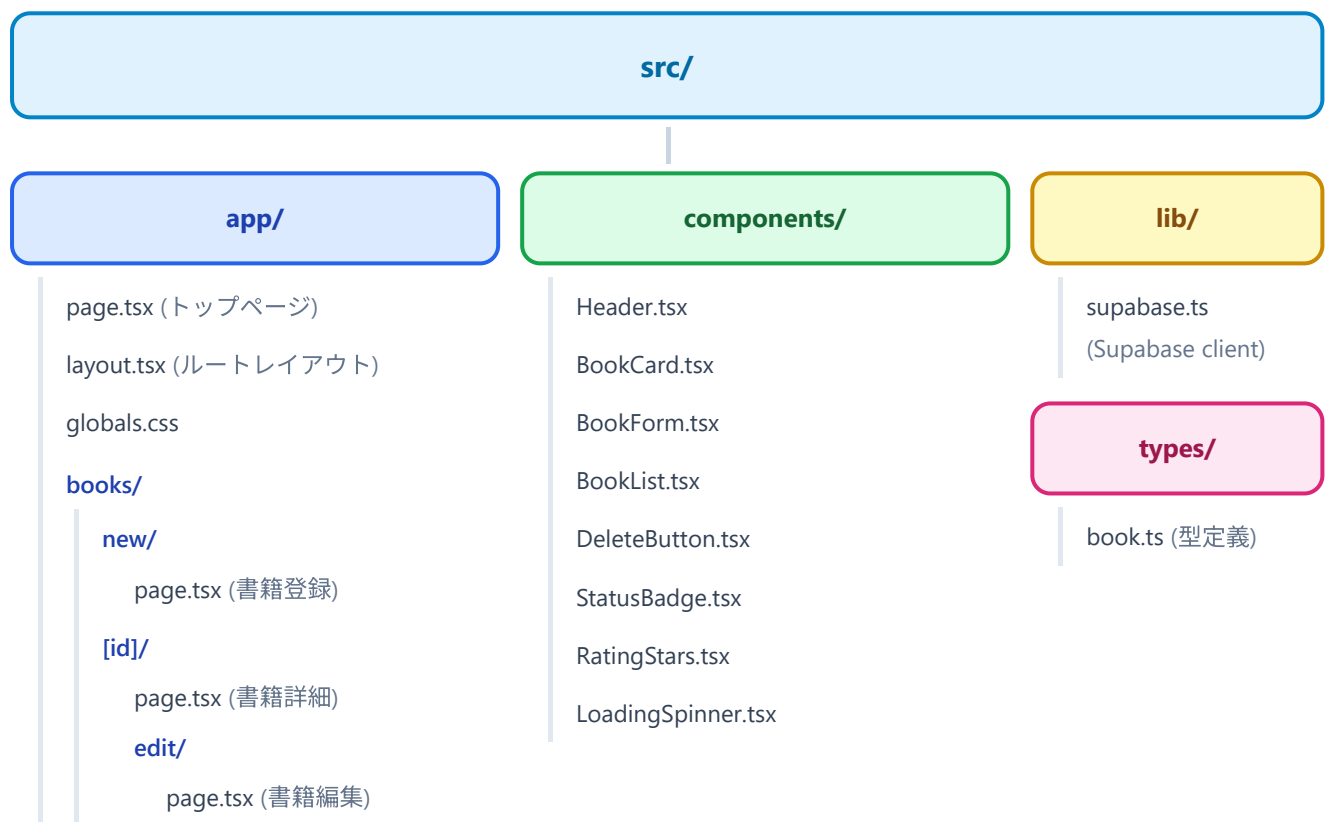
```
{
  "dependencies": {
    "@supabase/supabase-js": "^2.x.x",
    "next": "15.x.x",
    "react": "^19.x.x",
    "react-dom": "^19.x.x"
  }
}
```


^ の意味: **^2.49.1** は「メジャーバージョン 2 の範囲で最新版を許容する」という意味です。**2.49.1** から **2.99.99** まではインストール可能ですが、**3.0.0** はインストールされません。これにより、破壊的変更を避けつつ最新のバグ修正やパフォーマンス改善を受けられます。

4. プロジェクト構成の作成

4.1 ディレクトリ構成

書籍管理アプリでは、以下のディレクトリ構成を採用します。



テキスト形式でも示します。

```

src/
├─ app/
│   ├─ page.tsx          ... トップページ（書籍一覧を表示）
│   ├─ layout.tsx        ... ルートレイアウト（全ページ共通の枠組み）
│   ├─ globals.css       ... グローバル CSS
│   └─ books/
│       ├─ new/
│       │   └─ page.tsx  ... 書籍登録ページ
│       └─ [id]/
│           ├─ page.tsx  ... 書籍詳細ページ
│           └─ edit/
│               └─ page.tsx ... 書籍編集ページ
├─ components/
│   ├─ Header.tsx        ... 共通ヘッダー
│   ├─ BookCard.tsx      ... 書籍カード（一覧の各アイテム）
│   ├─ BookForm.tsx      ... 書籍登録・編集フォーム
│   ├─ BookList.tsx      ... 書籍一覧表示
│   ├─ DeleteButton.tsx  ... 削除ボタン
│   ├─ StatusBadge.tsx   ... ステータスバッジ
│   ├─ RatingStars.tsx   ... 評価（星）表示
│   └─ LoadingSpinner.tsx ... ローディング表示
├─ lib/
│   └─ supabase.ts       ... Supabase クライアント設定
└─ types/
    └─ book.ts           ... 型定義

```

4.2 各ファイルの役割

ページファイル（ `app/` ディレクトリ）

ファイル	URL パス	役割
<code>app/page.tsx</code>	<code>/</code>	トップページ。データベースから書籍の一覧を取得し、BookList コンポーネントで表示します。
<code>app/layout.tsx</code>	全ページ共通	ルートレイアウト。 <code><html></code> と <code><body></code> タグ、ヘッダーなどの全ページ共通要素を定義します。
<code>app/globals.css</code>	全ページ共通	Tailwind CSS のディレクティブとカスタムスタイルを定義します。
<code>app/books/new/page.tsx</code>	<code>/books/new</code>	書籍登録ページ。BookForm コンポーネントを使い、新しい書籍の情報を入力・保存します。
<code>app/books/[id]/page.tsx</code>	<code>/books/123</code>	書籍詳細ページ。URL の <code>[id]</code> 部分に書籍の ID が入り、その書籍の詳細情報を表示します。
<code>app/books/[id]/edit/page.tsx</code>	<code>/books/123/edit</code>	書籍編集ページ。既存の書籍情報を読み込み、BookForm コンポーネントで編集・更新します。

[id] とは？（動的ルーティング） フォルダ名を角括弧 `[]` で囲むと「動的ルート」になります。`/books/1`、`/books/2`、`/books/abc` のように、`[id]` の部分に任意の値が入ります。ページコンポーネントの `params` プロパティからその値を取得できます。

コンポーネントファイル（ **components/** ディレクトリ）

ファイル	役割
Header.tsx	アプリ上部に表示されるナビゲーションバー。アプリ名と「書籍を登録する」リンクを含みます。全ページで共通して表示されます。
BookCard.tsx	書籍一覧の中の1冊分の表示カード。タイトル、著者、ステータスバッジ、評価（星）を表示します。クリックすると詳細ページに遷移します。
BookForm.tsx	書籍の登録・編集に使うフォーム。タイトル、著者、ステータス、評価、メモの入力欄を持ちます。新規登録と編集の両方で共通して使用します。
BookList.tsx	BookCard を並べて一覧表示するコンポーネント。レスポンシブなグリッドレイアウトで配置します。書籍が0件の場合は「書籍が登録されていません」というメッセージを表示します。
DeleteButton.tsx	書籍を削除するボタン。クリック時に確認ダイアログを表示し、承諾した場合のみ削除を実行します。
StatusBadge.tsx	読書ステータス（「未読」「読書中」「読了」）を色付きバッジで表示するコンポーネント。ステータスに応じて色が変わります。
RatingStars.tsx	5段階の星評価を表示するコンポーネント。黄色い星と灰色の星で評価を視覚化します。
LoadingSpinner.tsx	データ読み込み中に表示するスピナー（回転アニメーション）。API からのレスポンスを待っている間、ユーザーに読み込み中であることを伝えます。

ユーティリティファイル（ **lib/** ディレクトリ）

ファイル	役割
supabase.ts	Supabase クライアントの初期化と設定。環境変数から Supabase の URL と API キーを読み取り、アプリ全体で利用できるクライアントインスタンスをエクスポートします。

型定義ファイル（ **types/** ディレクトリ）

ファイル	役割
book.ts	書籍データの TypeScript 型定義。 Book 型（データベースから取得する完全なデータ）、 BookInsert 型（新規登録時に送るデータ）、 BookUpdate 型（更新時に送るデータ）を定義します。

4.3 ディレクトリとファイルの作成

以下のコマンドで必要なディレクトリとファイルを一括作成します。

```
# ディレクトリの作成
mkdir -p src/app/books/new
mkdir -p src/app/books/[id]/edit
mkdir -p src/components
mkdir -p src/lib
mkdir -p src/types
```

注意: `[id]` は角括弧を含むため、シェルによってはエスケープが必要です。うまくいかない場合は、VS Code のエクスプローラーからフォルダを手動で作成してください。

```
# ファイルの作成（中身は後ほど記述）
touch src/components/Header.tsx
touch src/components/BookCard.tsx
touch src/components/BookForm.tsx
touch src/components/BookList.tsx
touch src/components/DeleteButton.tsx
touch src/components/StatusBadge.tsx
touch src/components/RatingStars.tsx
touch src/components/LoadingSpinner.tsx
touch src/lib/supabase.ts
touch src/types/book.ts
```

5. Supabase クライアントの設定

5.1 環境変数の取得

Supabase クライアントの設定には、以下の2つの情報が必要です。

1. Supabase URL -- あなたのプロジェクトの API エンドポイント
2. Supabase Anon Key -- 匿名アクセス用の API キー

これらは Supabase のダッシュボードから取得できます。

取得手順:

1. Supabase のダッシュボード にアクセスします
2. 書籍管理アプリ用のプロジェクトを選択します
3. 左のサイドバーから「Project Settings」（歯車アイコン）をクリックします
4. 「API」タブをクリックします
5. 以下の情報をコピーします: - Project URL: `https://xxxxxxxxxxxxxxxxx.supabase.co` の形式 - Project API keys の anon / public キー: `eyJhbGciOi...` で始まる長い文字列


```
// src/lib/supabase.ts

import { createClient } from "@supabase/supabase-js";

// 環境変数から Supabase の接続情報を取得
const supabaseUrl = process.env.NEXT_PUBLIC_SUPABASE_URL;
const supabaseAnonKey = process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY;

// 環境変数が設定されていない場合にわかりやすいエラーメッセージを表示
if (!supabaseUrl) {
  throw new Error(
    "環境変数 NEXT_PUBLIC_SUPABASE_URL が設定されていません。" +
    ".env.local ファイルを確認してください。"
  );
}

if (!supabaseAnonKey) {
  throw new Error(
    "環境変数 NEXT_PUBLIC_SUPABASE_ANON_KEY が設定されていません。" +
    ".env.local ファイルを確認してください。"
  );
}

// Supabase クライアントを作成してエクスポート
// アプリ全体で同じインスタンスを使い回す（シングルトンパターン）
export const supabase = createClient(supabaseUrl, supabaseAnonKey);
```

コードの解説:

行	解説
<code>import { createClient } from "@supabase/supabase-js";</code>	Supabase の公式ライブラリから <code>createClient</code> 関数をインポートします。この関数が Supabase との接続を確立します。
<code>process.env.NEXT_PUBLIC_SUPABASE_URL</code>	<code>.env.local</code> に定義した環境変数を読み取ります。 <code>process.env</code> は Node.js が環境変数にアクセスするためのオブジェクトです。
<code>if (!supabaseUrl)</code>	環境変数が未設定（ <code>undefined</code> ）の場合にエラーを投げます。 <code>.env.local</code> の作成忘れや typo を早期に検出できます。
<code>createClient(supabaseUrl, supabaseAnonKey)</code>	Supabase クライアントを生成します。第1引数が API の URL、第2引数が認証キーです。
<code>export const supabase</code>	生成したクライアントを <code>supabase</code> という名前でエクスポートします。他のファイルから <code>import { supabase } from "@lib/supabase";</code> でインポートして使います。

6. 型定義ファイルの作成

6.1 types/book.ts の作成

`src/types/book.ts` ファイルに以下のコードを記述します。この型定義は、アプリ全体で書籍データの構造を統一するために使います。


```
// src/types/book.ts

/**
 * 読書ステータスの型定義
 *
 * - "unread" : 未読（まだ読んでいない）
 * - "reading" : 読書中（今読んでいる途中）
 * - "finished" : 読了（読み終わった）
 */
export type BookStatus = "unread" | "reading" | "finished";

/**
 * 書籍データの型定義（データベースから取得した完全なデータ）
 *
 * Supabase の books テーブルの行データに対応します。
 * すべてのフィールドが含まれており、データベースから取得した
 * 書籍データはこの型に一致します。
 */
export type Book = {
  /** 書籍ID (UUID形式、Supabase が自動生成) */
  id: string;

  /** 書籍のタイトル（必須） */
  title: string;

  /** 著者名（必須） */
  author: string;

  /** 読書ステータス（デフォルト: "unread") */
  status: BookStatus;

  /** 評価（1～5の整数、未評価の場合は null） */
  rating: number | null;

  /** メモ・感想（任意、未入力の場合は null） */
  memo: string | null;

  /** 作成日時 (ISO 8601 形式の文字列、Supabase が自動生成) */
  created_at: string;

  /** 更新日時 (ISO 8601 形式の文字列、Supabase が自動生成) */
  updated_at: string;
};

/**
 * 書籍の新規登録時に送るデータの型定義
 *
 * id, created_at, updated_at はデータベースが自動生成するため、
 * 送信データには含めません。
 */

```

```

* status, rating, memo はオプション (省略可能) です。
*/
export type BookInsert = {
  /** 書籍のタイトル (必須) */
  title: string;

  /** 著者名 (必須) */
  author: string;

  /** 読書ステータス (省略時は "unread" がデフォルト) */
  status?: BookStatus;

  /** 評価 (1~5、省略時は null) */
  rating?: number | null;

  /** メモ・感想 (省略時は null) */
  memo?: string | null;
};

/**
 * 書籍の更新時に送るデータの型定義
 *
 * 更新したいフィールドだけを送ればよいため、
 * すべてのフィールドがオプションです。
 * Partial<BookInsert> と同じ意味ですが、明示的に定義しています。
 */
export type BookUpdate = {
  /** 書籍のタイトル */
  title?: string;

  /** 著者名 */
  author?: string;

  /** 読書ステータス */
  status?: BookStatus;

  /** 評価 (1~5、null で評価をクリア) */
  rating?: number | null;

  /** メモ・感想 (null でメモをクリア) */
  memo?: string | null;
};

/**
 * ステータスの表示ラベル定義
 *
 * コンポーネントでステータスを日本語表示するときに使います。
 * 例: statusLabels["reading"] → "読書中"
 */
export const statusLabels: Record<BookStatus, string> = {

```

```

unread: "未読",
reading: "読書中",
finished: "読了",
};

/**
 * ステータスに対応する色のクラス定義
 *
 * StatusBadge コンポーネントでバッジの色を切り替えるときに使います。
 * Tailwind CSS のクラス名を直接指定しています。
 */
export const statusColors: Record<
  BookStatus,
  { bg: string; text: string }
> = {
  unread: {
    bg: "bg-gray-100",
    text: "text-gray-700",
  },
  reading: {
    bg: "bg-blue-100",
    text: "text-blue-700",
  },
  finished: {
    bg: "bg-green-100",
    text: "text-green-700",
  },
};

```

コードの解説:

型 / 定数	用途
<code>BookStatus</code>	読書ステータスを3つのリテラル型のユニオンで定義します。 <code>"unread" "reading" "finished"</code> 以外の値を代入しようとすると、TypeScript がコンパイルエラーを出します。
<code>Book</code>	データベースの <code>books</code> テーブルの1行に対応する型です。 <code>supabase.from("books").select("*")</code> の戻り値はこの型の配列になります。
<code>BookInsert</code>	新規登録時に必要なデータの型です。 <code>title</code> と <code>author</code> が必須、それ以外はオプション（ <code>?</code> 付き）です。 <code>id</code> や <code>created_at</code> はデータベースが自動で付与するため含みません。
<code>BookUpdate</code>	更新時に送るデータの型です。すべてのフィールドがオプションで、変更したい項目だけを送ります。
<code>statusLabels</code>	ステータスの文字列を日本語ラベルに変換するためのマッピングオブジェクトです。 <code>Record<BookStatus, string></code> は「キーが <code>BookStatus</code> 型、値が <code>string</code> 型のオブジェクト」を意味します。
<code>statusColors</code>	ステータスに対応する Tailwind CSS のクラス名をマッピングしたオブジェクトです。StatusBadge コンポーネントでバッジの背景色と文字色を動的に切り替えるために使います。

なぜ3つの型を分けるのか？

データベースとやり取りする場面によって、必要なフィールドが異なります。

- 取得（SELECT）：すべてのフィールドが返ってくる → `Book` 型
- 登録（INSERT）：`id` や `created_at` は不要 → `BookInsert` 型
- 更新（UPDATE）：変更したいフィールドだけ送る → `BookUpdate` 型

型を分けることで、各場面で正確なデータ構造を強制でき、バグを防げます。

7. ルートレイアウトの設定

7.1 app/layout.tsx の作成

`src/app/layout.tsx` はすべてのページを囲む「外枠」のようなものです。HTML の `<html>` と `<body>` タグを含み、全ページ共通のヘッダーや CSS をここで設定します。

以下のコードで `src/app/layout.tsx` を書き換えます（`create-next-app` が生成したデフォルトの内容を完全に置き換えます）。

```
// src/app/layout.tsx

import type { Metadata } from "next";
import { Noto_Sans_JP } from "next/font/google";
import "./globals.css";
import Header from "@/components/Header";

/**
 * フォント設定
 *
 * Google Fonts から Noto Sans JP（日本語対応フォント）を読み込みます。
 * next/font/google を使うことで、フォントファイルがビルド時に
 * 自動的に最適化・セルフホスティングされます。
 * （外部の Google Fonts サーバーにリクエストが飛ばないため高速）
 */
const notoSansJP = Noto_Sans_JP({
  subsets: ["latin"],
  weight: ["400", "500", "700"],
  display: "swap",
  preload: true,
});

/**
 * メタデータ設定
 *
 * ブラウザのタブに表示されるタイトルや、
 * 検索エンジンに表示される説明文を設定します。
 */
export const metadata: Metadata = {
  title: "書籍管理アプリ",
  description: "読んだ本、読んでいる本、これから読む本を管理するWebアプリケーション",
};

/**
 * ルートレイアウトコンポーネント
 *
 * すべてのページはこのレイアウトの中にレンダリングされます。
 * { children } には各ページのコンテンツが入ります。
 *
 * 構造:
 * <html>
 *   <body>
 *     <Header /> ← 全ページ共通のヘッダー
 *     <main>{children}</main> ← 各ページの内容
 *   </body>
 * </html>
 */
export default function RootLayout({
```

```

    children,
  }: Readonly<{
    children: React.ReactNode;
  }> {
    return (
      <html lang="ja">
        <body className={` ${notoSansJP.className} bg-gray-50 min-h-screen`} >
          <Header />
          <main className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">
            {children}
          </main>
        </body>
      </html>
    );
  }

```

コードの解説:

部分	解説
<code>import type { Metadata } from "next";</code>	Next.js のメタデータ型をインポートします。 <code>type</code> キーワードは「型のみインポート」を意味し、ランタイムには影響しません。
<code>import { Noto_Sans_JP } from "next/font/google";</code>	Google Fonts から Noto Sans JP フォントをインポートします。 <code>next/font</code> を使うと、フォントがビルド時にダウンロードされ、パフォーマンスが最適化されます。
<code>import "./globals.css";</code>	Tailwind CSS のグローバルスタイルを読み込みます。この1行がないと Tailwind のクラスが機能しません。
<code>import Header from "@components/Header";</code>	共通ヘッダーコンポーネントをインポートします。 <code>@/</code> は <code>src/</code> ディレクトリのエイリアスです。
<code>subsets: ["latin"]</code>	ラテン文字のサブセットを読み込みます。日本語文字はデフォルトで含まれます。
<code>weight: ["400", "500", "700"]</code>	使用するフォントウェイト（太さ）を指定します。400 = 通常、500 = やや太字、700 = 太字。
<code>display: "swap"</code>	フォントの読み込み方を指定します。 <code>"swap"</code> はフォント読み込み完了前にシステムフォントで表示し、読み込み後に切り替えます（FOITを防ぐ）。
<code>export const metadata</code>	ページのメタデータを定義します。 <code>title</code> がブラウザのタブに表示され、 <code>description</code> が検索エンジンの説明文に使われます。
<code>lang="ja"</code>	HTML の言語を日本語に設定します。スクリーンリーダーや検索エンジンが正しく言語を認識できます。
<code>bg-gray-50</code>	<code><body></code> の背景色をごく薄いグレーにします。真っ白よりも目に優しく、カードなどの白い要素が際立ちます。
<code>min-h-screen</code>	<code><body></code> の最小高さを画面いっぱいにします。コンテンツが少ないページでも背景色が画面全体に適用されます。
<code>max-w-7xl mx-auto</code>	<code><main></code> の最大幅を 1280px に制限し、左右の margin を auto にして中央寄せにします。大画面でもコンテンツが横に広がりすぎるのを防ぎます。
<code>px-4 sm:px-6 lg:px-8</code>	画面サイズに応じて左右の padding を変えます。モバイルでは 16px、タブレットでは 24px、PCでは 32px です。
<code>{children}</code>	各ページのコンテンツがここに挿入されます。 <code>/</code> にアクセスすれば <code>app/page.tsx</code> の内容が、 <code>/books/new</code> にアクセスすれば <code>app/books/new/page.tsx</code> の内容が入ります。

7.2 globals.css の確認

`src/app/globals.css` は、`create-next-app` がデフォルトで生成した内容をそのまま使用します。以下の Tailwind CSS ディレクティブが含まれていることを確認してください（不要なデフォルトスタイルは削除して構いません）。

```
/* src/app/globals.css */

@tailwind base;
@tailwind components;
@tailwind utilities;
```

Tailwind CSS ディレクティブの意味:

ディレクティブ	役割
<code>@tailwind base;</code>	ブラウザ間の表示差異をなくすリセット CSS と、基本的な要素のスタイルを注入します。
<code>@tailwind components;</code>	コンポーネントクラス（ <code>container</code> など）を注入します。
<code>@tailwind utilities;</code>	ユーティリティクラス（ <code>flex</code> , <code>pt-4</code> , <code>text-center</code> など）をすべて注入します。未使用のクラスはビルド時に自動削除されます。

Next.js のバージョンによる違い: Next.js 15 以降のバージョンでは、`globals.css` の記法が `@import "tailwindcss";` のように変わっている場合があります。 `create-next-app` が生成した内容がどちらの形式であっても、Tailwind CSS は正しく動作しますのでそのまま使用してください。

8. 共通ヘッダーコンポーネント

8.1 components/Header.tsx の作成

`src/components/Header.tsx` ファイルに以下のコードを記述します。


```
// src/components/Header.tsx

"use client";

import Link from "next/link";
import { usePathname } from "next/navigation";

/**
 * 共通ヘッダーコンポーネント
 *
 * すべてのページの上部に表示されるナビゲーションバーです。
 * - アプリ名（クリックでトップページに遷移）
 * - 「書籍を登録する」ボタン
 * を含みます。
 *
 * "use client" を使用する理由:
 * usePathname() フックを使用するため、Client Component にする必要があります。
 * Server Component では React のフック (useState, useEffect, usePathname 等) は使えません。
 */
export default function Header() {
  // 現在のURLパスを取得（アクティブなナビゲーションリンクのスタイル変更で使用）
  const pathname = usePathname();

  return (
    <header className="bg-white shadow-sm border-b border-gray-200">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
        <div className="flex items-center justify-between h-16">
          {/* アプリ名 (ロゴ) */}
          <Link
            href="/"
            className="text-xl font-bold text-gray-900 hover:text-blue-600 transition duration-200"
          >
            書籍管理アプリ
          </Link>

          {/* ナビゲーション */}
          <nav className="flex items-center space-x-4">
            {/* 書籍一覧リンク */}
            <Link
              href="/"
              className={`px-3 py-2 rounded-md text-sm font-medium transition duration-200
                pathname === "/"
                  ? "bg-blue-50 text-blue-700"
                  : "text-gray-600 hover:text-gray-900 hover:bg-gray-50"
              }`
            >
              書籍一覧
            </Link>

```

```

    { /* 書籍登録ボタン */ }
    <Link
      href="/books/new"
      className={`px-4 py-2 rounded-md text-sm font-medium transition duration-200
        pathname === "/books/new"
          ? "bg-blue-700 text-white"
          : "bg-blue-600 text-white hover:bg-blue-700"
        `}
    >
      書籍を登録する
    </Link>
  </nav>
</div>
</div>
</header>
);
}

```

コードの解説:

部分	解説
<code>"use client";</code>	このファイルを Client Component として宣言します。Next.js の App Router では、デフォルトですべてのコンポーネントが Server Component（サーバーで実行）です。しかし <code>usePathname()</code> などの React フックを使うには Client Component にする必要があります。ファイルの先頭にこの宣言を書くだけで切り替えられます。
<code>import Link from "next/link";</code>	Next.js のリンクコンポーネントです。HTML の <code><a></code> タグの代わりに使います。ページ遷移時にページ全体をリロードせず、必要な部分だけを更新するため高速です。
<code>import { usePathname } from "next/navigation";</code>	現在のURLパスを取得するフックです。例えば <code>/books/new</code> にいるときは <code>"/books/new"</code> が返ります。
<code>pathname === "/"</code>	現在のページがトップページかどうかを判定し、条件に応じてスタイルを切り替えます。現在いるページのリンクには背景色を付けてハイライト表示します。
<code>bg-white shadow-sm border-b border-gray-200</code>	ヘッダーの背景を白にし、下に薄いシャドウとボーダーを付けてコンテンツ部分との区切りを明確にします。
<code>max-w-7xl mx-auto</code>	ヘッダー内のコンテンツの最大幅を 1280px にし、中央寄せにします。 <code>layout.tsx</code> の <code><main></code> と同じ幅にすることで整った見た目になります。
<code>flex items-center justify-between h-16</code>	Flexbox で子要素を横並びにし、垂直方向に中央揃え、左右に均等配置、高さ64pxを設定します。
<code>space-x-4</code>	ナビゲーションリンク間に 16px の水平方向の間隔を設けます。

9. 開発サーバーの起動と動作確認

9.1 仮のトップページを作成

動作確認のため、`src/app/page.tsx` を以下の内容に書き換えます（`create-next-app` のデフォルト内容を置き換えます）。

```
// src/app/page.tsx

export default function HomePage() {
  return (
    <div>
      <h1 className="text-3xl font-bold text-gray-900 mb-4">
        書籍一覧
      </h1>
      <p className="text-gray-600 mb-8">
        登録された書籍がここに表示されます。
      </p>

      {/* 動作確認用: Tailwind CSS のスタイルが正しく適用されるかチェック */}
      <div className="bg-white rounded-lg shadow-sm border border-gray-200 p-6">
        <h2 className="text-lg font-semibold text-gray-800 mb-2">
          セットアップ完了
        </h2>
        <p className="text-gray-600 text-sm">
          Next.js + Tailwind CSS + Supabase の開発環境が正しくセットアップされました。
          次の章で、このページに実際の書籍一覧を表示します。
        </p>
        <div className="mt-4 flex space-x-2">
          <span className="inline-block bg-gray-100 text-gray-700 text-xs px-2.5 py-1 rounded">
            未読
          </span>
          <span className="inline-block bg-blue-100 text-blue-700 text-xs px-2.5 py-1 rounded">
            読書中
          </span>
          <span className="inline-block bg-green-100 text-green-700 text-xs px-2.5 py-1 rounded">
            読了
          </span>
        </div>
      </div>
    </div>
  );
}
```

9.2 開発サーバーの起動

ターミナルで以下のコマンドを実行します。

```
npm run dev
```

以下のような出力が表示されます。

```
▲ Next.js 15.x.x (Turbopack)
- Local:      http://localhost:3000
- Network:    http://192.168.x.x:3000

✓ Starting...
✓ Ready in xxxms
```

9.3 ブラウザでの確認

ブラウザで `http://localhost:3000` を開くと、以下のように表示されます。

ヘッダー部分: - 画面最上部に白い背景のヘッダーバーが表示されます - 左側に「書籍管理アプリ」というテキストが太字で表示されます - 右側に「書籍一覧」というテキストリンクと「書籍を登録する」という青いボタンが表示されます - 「書籍一覧」リンクは現在のページなので、薄い青の背景でハイライトされています

メインコンテンツ部分: - ヘッダーの下に薄いグレー（`bg-gray-50`）の背景があります - 「書籍一覧」という大きな見出し（30px, 太字）が表示されます - その下に「登録された書籍がここに表示されます。」という説明文があります - 白い角丸のカード内に「セットアップ完了」というメッセージが表示されます - カードの下部に3つのステータスバッジ（「未読」はグレー、「読書中」は青、「読了」は緑）が横並びで表示されます

確認ポイント:

確認項目	期待される状態
ヘッダーが表示される	白い背景のバーが画面上部にある
Tailwind CSS が適用されている	テキストに色やサイズが適用され、カードに角丸やシャドウがある
フォント（Noto Sans JP）が適用されている	日本語テキストが美しいフォントで表示される
レスポンシブ対応	ブラウザの幅を狭めても、レイアウトが崩れない
ナビゲーションリンクが動作する	「書籍を登録する」ボタンをクリックすると <code>/books/new</code> に遷移する（まだページを作成していないため 404 が表示されますが、URL が変わることを確認）

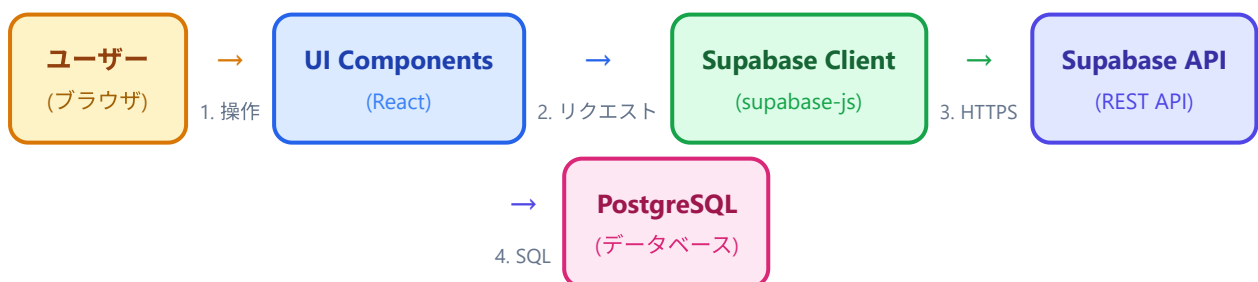
9.4 よくあるトラブルと対処法

症状	原因	対処法
<code>Module not found: Can't resolve '@components/Header'</code>	ファイルが存在しない、またはパスが間違っている	<code>src/components/Header.tsx</code> が正しい場所に作成されているか確認してください。ファイル名の太文字小文字も一致している必要があります。
スタイルが全く適用されない	<code>globals.css</code> が正しく読み込まれていない	<code>src/app/layout.tsx</code> で <code>import './globals.css';</code> が記述されているか確認してください。
環境変数 <code>NEXT_PUBLIC_SUPABASE_URL</code> が設定されていません	<code>.env.local</code> が未作成または場所が間違っている	<code>.env.local</code> がプロジェクトのルートディレクトリ（ <code>package.json</code> と同じ階層）に存在するか確認してください。ファイル作成後は開発サーバーの再起動（ <code>Ctrl+C</code> で停止してから <code>npm run dev</code> ）が必要です。
ポート 3000 が既に使用されている	他のアプリが 3000 番を使っている	<code>npm run dev -- --port 3001</code> で別のポートを指定するか、使用中のアプリを停止してください。
日本語フォントが適用されていない	フォントの読み込みに失敗している	ネットワーク接続を確認してください。初回ビルド時に Google Fonts からフォントがダウンロードされます。

10. データフローの全体像

書籍管理アプリのデータの流れを図で確認しましょう。ユーザーの操作がどのようにデータベースに伝わり、結果がどのように画面に戻ってくるのかを理解することは、アプリ全体の設計を把握する上で非常に重要です。

10.1 全体アーキテクチャ



戻りの流れ: 5. クエリ結果 ← 6. JSON レスポンス ← 7. 型付きデータ (Book[]) ← 8. 画面更新 (再レンダリング)

10.2 各ステップの詳細解説

ステップ	場所	処理内容
1. ユーザーの操作	ブラウザ	ユーザーがボタンをクリックしたり、フォームに入力したりします。例: 「書籍を登録する」ボタンを押す。
2. コンポーネントからの呼び出し	UI Components	React コンポーネント内で Supabase クライアントの関数を呼び出します。例: <code>supabase.from("books").insert(newBook)</code>
3. HTTP リクエスト	Supabase Client	<code>@supabase/supabase-js</code> ライブラリが、JavaScript のコードを HTTPS リクエストに変換して Supabase のサーバーに送信します。URL や Anon Key は <code>.env.local</code> の環境変数から読み取られます。
4. SQL クエリの実行	Supabase API	Supabase の REST API (PostgREST) がリクエストを受け取り、対応する SQL クエリに変換して PostgreSQL データベースに実行します。RLS (行レベルセキュリティ) によるアクセス制御もこのタイミングで適用されます。
5. クエリ結果	PostgreSQL	データベースがクエリを実行し、結果を返します。
6. JSON レスポンス	Supabase API	クエリ結果を JSON 形式に変換し、クライアントにレスポンスとして返します。
7. 型付きデータ	Supabase Client	JSON レスポンスを受け取り、TypeScript の型 (<code>Book[]</code> など) に変換します。
8. 画面更新	UI Components → ブラウザ	受け取ったデータで React の state を更新し、画面が再レンダリングされます。ユーザーは最新のデータを画面上で確認できます。

10.3 具体例: 書籍一覧の取得

実際のコードでデータがどう流れるか、具体例で見てみましょう (このコードは次の章で実装します)。

```
// 書籍一覧を取得するコードの流れ

// 1. Supabase クライアントをインポート
import { supabase } from "@lib/supabase";
import { Book } from "@types/book";

// 2. データベースから書籍一覧を取得
const { data, error } = await supabase
  .from("books")           // books テーブルを指定
  .select("*")             // すべてのカラムを取得
  .order("created_at", {   // 作成日の降順（新しい順）に並び替え
    ascending: false
  });

// 3. エラーハンドリング
if (error) {
  console.error("書籍の取得に失敗しました:", error.message);
  return;
}

// 4. data は Book[] 型（書籍の配列）
const books: Book[] = data;
```

この1つの `supabase.from("books").select("*")` の呼び出しの裏側で、上の図の 2 → 3 → 4 → 5 → 6 → 7 のステップがすべて自動的に実行されています。Supabase クライアントがこれらの複雑な処理を抽象化してくれるため、開発者は SQL を直接書くことなく、シンプルな JavaScript/TypeScript のコードでデータベースを操作できます。

まとめ

この章で実施した内容を振り返ります。

項目	内容
Next.js プロジェクトの作成	<code>create-next-app</code> で TypeScript + Tailwind CSS + App Router のプロジェクトを作成しました。
Tailwind CSS の基礎	ユーティリティファーストの概念と、よく使うクラスを学びました。
パッケージのインストール	<code>@supabase/supabase-js</code> をインストールしました。
ディレクトリ構成の設計	<code>app/</code> （ページ）、 <code>components/</code> （UI部品）、 <code>lib/</code> （ユーティリティ）、 <code>types/</code> （型定義）の4つのディレクトリで整理する構成にしました。
Supabase クライアントの設定	<code>.env.local</code> に環境変数を設定し、 <code>lib/supabase.ts</code> でクライアントを初期化しました。
型定義の作成	<code>Book</code> 、 <code>BookInsert</code> 、 <code>BookUpdate</code> の3つの型と、ステータス関連の定数を定義しました。
ルートレイアウトの設定	日本語フォント、メタデータ、全ページ共通のレイアウト構造を設定しました。
ヘッダーコンポーネントの作成	ナビゲーション付きの共通ヘッダーを作成しました。
動作確認	開発サーバーを起動し、ブラウザで表示を確認しました。

作成したファイル一覧:

src/	
├─ app/	
│ ├─ layout.tsx	✓ 作成済み
│ ├─ page.tsx	✓ 作成済み（仮の内容）
│ └─ globals.css	✓ 確認済み
├─ components/	
│ └─ Header.tsx	✓ 作成済み
├─ lib/	
│ └─ supabase.ts	✓ 作成済み
└─ types/	
└─ book.ts	✓ 作成済み

次の章では、書籍一覧の表示機能を実装します。`BookCard`、`BookList`、`StatusBadge`、`RatingStars`、`LoadingSpinner` の各コンポーネントを作成し、Supabase からデータを取得してトップページに表示します。